Agentic AI Nutritionist

Tyler Blue and Andrew Moy
Georgetown University, DSAN 6725, Spring 2026
Professor Amit Arora
April 24, 2026

Abstract

Families and individuals increasingly struggle to consistently plan and prepare meals due to busy professional and personal lives. Many existing apps and websites exist to help solve this problem by automatically generating meal plans and grocery lists. This paper presents a meal planning application that integrates an agentic AI infrastructure that enables plan generation through natural language and persistent user memory. The system uses a three-agent architecture built on the Strands Agent SDK with Claude language models. User profiles are stored across sessions via Mem0, and personalized meal plans are created from 751 webscraped recipes. The system is evaluated on 29 test cases across eight deterministic scorers and achieved perfect scores on allergen safety, error rate, and shopping list correctness, with plan completeness and tool routing both exceeding 0.97. Calorie target adherence and latency scored lower but these scores are a result of architecture design rather than system error, demonstrating that an agentic approach to personalized meal planning is feasible.

Introduction

For most of human history, food was an extremely scarce resource – a vast majority of our time was spent hunting and gathering to prepare our next meal. Today, the dynamic has done a complete reversal – in a world where food is abundant and accessible, free time is the scarce resource, and busy professionals, parents, and others often struggle to find the time to grocery shop and prepare meals consistently.

Meal planning is a strategy used by many to reduce the cognitive load that comes with preparing food, as well as the time it takes to plan out a meal. Many apps such as "EatThisMuch" and "Mealime" further help users to plan out their weeks, and while these apps are effective, there is potential to significantly upgrade these apps through the integration of large language models. Such integration would reduce the friction required to use the app by allowing users to interface with it through natural language. However, building such a system is non-trivial – it requires an agentic AI architecture from which the app can determine intent, select and use tools, and interact with the underlying structure of the app. This project investigates the design and implementation of such an agentic AI meal planning application.

Related Work

Early personalized nutrition systems relied on rule-based algorithms over manually curated recipe databases. While effective, these approaches suffer from high complexity and limited

adaptability. Khamesian et al. [NutriGen] note that existing systems "often lack adaptability, fail to consider real-world constraints such as food ingredient availability, and require extensive user input." Papastratis et al. [2024] address this by combining a variational autoencoder with ChatGPT for meal variety. LLM-based systems reduce this complexity but also introduce accuracy concerns – Guo et al. [2025] reviewed 23 studies on ChatGPT's meal planning performance, and although accuracy was comparable to licensed dieticians, safety risks and hallucinations were common. Bilen et al. [2026] found that five leading LLMs undercalculated daily energy intake by about 695 calories on average compared to the plans of dieticians, showing that AI-generated dietary recommendations are not reliable over human supervision. This motivates our decision to assemble plans from recipes with pre-validated nutritional profiles rather than generating them freely.

Research has also been done related to the conversational side of meal planning – Angara et al. [2017] established an early blueprint with Foodie Fooderson, a Watson-based kitchen assistant that combined natural language understanding with context-aware recipe retrieval. Our system extends this with persistent cross-session memory and multi-day planning. In another study, Abbas et al. [CHI 2026] found that users mistrust agents who forget past conversations – this backs up our choice to use an “intake agent” that stores profile memory across sessions.

Commercial services like MyFitnessPal and Mealime rely on manual data entry with little cross-session personalization. Newer agentic tools like Dinnerbell and Nori represent the more automated end of the spectrum. Our project combines these concepts by combining meal plan generation integrated with conversational interaction methods.

Data

The system provides a set of base recipes provided by Budget Bytes. Budget Bytes was selected because we wanted to include cost as a decision making tool for the agent, and each recipe on the website includes a total cost and a cost-per-serving figure. Recipe data was collected through web scraping using Python’s ‘requests’ and ‘BeautifulSoup’ libraries.

In contrast to the cost values, the nutritional information in this database — calories, protein, fat, and carbohydrates per serving — was estimated using a large language model rather than calculated from an official nutritional database. This was intended to speed up development and

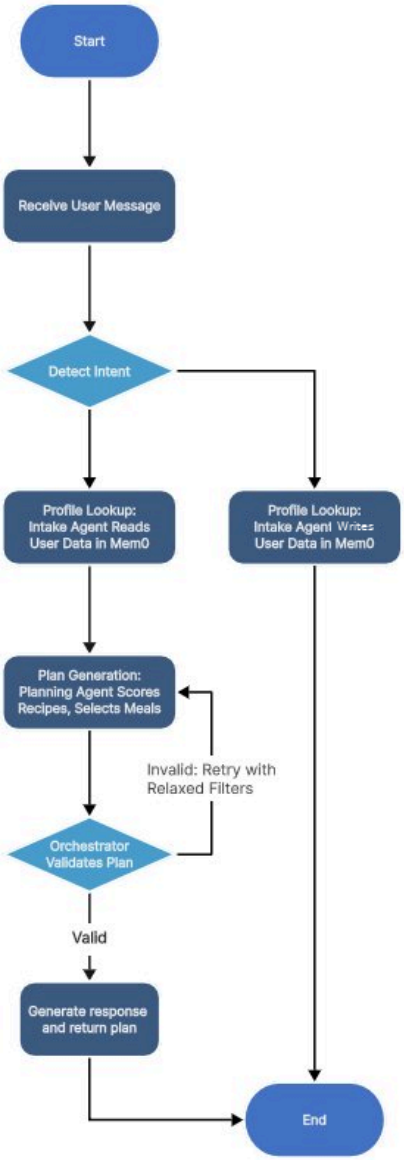
testing. The estimates are fairly reasonable, but inaccurate nutrition facts introduce an obvious problem for real users.

Each recipe was also assigned a Budget Bytes category label retrieved from the WordPress REST API during scraping. This label is what the planning agent uses to route recipes to the correct meal slot at runtime — breakfast recipes go to the breakfast slot, main dishes to lunch and dinner, and sides, snacks, and desserts to side slots. The final dataset contains 751 recipes with a median cost per serving of $1.22. These recipes are saved to a personal SQLite database created for each user, which the user can edit, add to, and remove from.

Category	Count	Slot
Main Dish	508	Lunch / Dinner
Side Dish	95	Side
Breakfast	66	Breakfast
Dessert	54	Side
Snack	22	Side
Drink	6	Side
Total	**751**	

System Architecture

The overarching architecture of this system involves a central Orchestrator Agent and two sub agents, the Intake Agent and the Planning Agent. The Orchestrator serves as the user facing agent and routes tasks to two sub-agents that handle distinct domains – the Intake Agent manages user profile data, and the Planning Agent builds personalized meal plans. Agents use different Claude LLM models to operate depending on the complexity of their task.



1. ***Orchestrator Agent:***

Model: Claude Sonnet 4.6

The Orchestrator is responsible for interfacing with the user and routing their messages. This agent analyzes each message, determines its intent, and routes it to the appropriate subagent or tool. The orchestrator is also responsible for cookbook management – it identifies a task related to the recipe database and performs the one shot CRUD operation via a tool, rather than routing to a 'Cookbook Agent', because no additional reasoning is required.

Tool	*Description*
update_user_profile	Delegates profile-related messages to the Intake Agent.
build_meal_plan	Delegates meal plan requests to the Planning Agent.
audit_meal_plan	Deterministically checks a plan against constraints (allergens, # of meals, calories)
generate_shopping_list	Aggregates all ingredients across a plan
add_recipe_to_cookbook	Adds a user-described recipe. Any missing nutrition fields are estimated via a direct Claude API call.
add_recipe_from_url	Scrapes a recipe from a URL, estimates missing fields, and saves it to the cookbook.
edit_cookbook_recipe	Updates specific fields on an existing custom recipe.

Intake Agent:

Model: Claude Sonnet 4.6

The Intake Agent manages a user's profile, identified through a name that serves as a user id. Deployed versions of this application would integrate authentication into this agent. The agent collects and updates user profile data through natural conversation, asking one or two questions at a time to avoid overwhelming the user. Profile data is stored using Mem0, a cloud vector memory database. Each fact is stored as a single memory entry — when a field is updated, the old entry is deleted and replaced with the new information.

Tool	*Description*

get_user_profile	Loads all memories for the user
update_goal	Stores the user's nutrition goal (fat loss, muscle gain, or maintenance)
update_body_stats	Stores weight, height, age, and sex.
update_activity_level	Stores activity level on a scale from sedentary to very active.
update_dietary_preferences	Stores allergies, disliked ingredients, and preferred meal categories
update_meal_constraints	Stores maximum cost per serving, maximum cook time, maximum ingredient count, and meals per day.
recall_user_info	Performs a semantic search over stored memories to answer a specific question about the user.
reset_user_profile	Deletes all memories for the user.
remove_cookbook_recipe	Hides base recipes from planning or permanently deletes custom recipes.

Planning Agent:

Model: Claude Haiku 4.5

The planning agent is used for the core functionality of the app – creating meal plans. It reads the user profile to compute cost limits and calorie/protein targets (using the Mifflin-St Jeor formula), and then reads the recipe dataset to curate an appropriate plan for the selected number of days.

Tool	***Description***
get_user_profile	Reads the user's profile from Mem0
estimate_calorie_target	Computes the daily calorie target using user profile

	information and the Mifflin-St Jeor formula
estimate_protein_target	Computes the daily protein target based on goal and body weight.
build_day_plan_tool	Selects one day of meals scored against calorie and protein targets
build_week_plan_tool	Selects meals for N days
search_meals_tool	Searches the recipe dataset by meal slot, filters, and keyword.

This infrastructure is stateless, with profile information stored on a persistent cloud vector database (Mem0) and recipes stored on a local file via SQLite. Only the previous chat history is stored within each session.

Models and Technologies

1. Mem0

Mem0 is a cloud vector database used to store persistent user profile information in natural language based on a user's name/id. Semantic search is used to retrieve and overwrite relevant information to the user. Mem0 handles vector embeddings internally, so that functionality is abstracted away. Mem0 was chosen over a local vector store because it is a free, simple to use service with no overhead that can be accessed via an API.

2. Strands Agent SDK

Agent orchestration is implemented using the Strands Agent SDK. Strands was chosen over alternatives such as LangChain because it provides a simple pipeline for tool and agent definitions, making the system easier to design and comprehend. Each agent is defined by a model, a system prompt, and an explicit list of Python functions registered as tools — the SDK handles everything else.

3. Streamlit

The user-facing interface is a Streamlit web application, chosen for its ease of use as a demo. A deployed application would require a restructuring of the tech stack.

4. SQLite

Recipes are stored in a SQLite cookbook, stored locally on the user's computer. Similarly, a cloud database would likely be implemented for a production version of the product. When required, an agent runs a local SQL query at runtime to find the user's recipes.

5. Direct LLM Calls

Two tools in the architecture make direct API calls – nutrition estimation and url recipe extraction. This is because these are structured tasks that do not require reasoning by an agent – only the output is needed.

Infrastructure and deployment considerations

The software engineering aspect of the app runs locally with no remote servers – Streamlit provides the frontend, and SQLite stores the cookbook on disk. The AI agents do require internet connection, as they run via the Anthropic API and the Mem0 cloud vector store.

Several design decisions were made deliberately to minimize cost and latency (via high cost Anthropic API calls). The Planning Agent uses Claude Haiku rather than Sonnet, since meal plan generation follows a structured workflow that doesn't need higher level reasoning. Recipe selection could be solved by an LLM, but a deterministic scoring algorithm is used instead, with penalties for nutrition values and preferences.

The multi-agent architecture is also important to isolate functionality of the system – having a single general agent monitor the entire app is feasible, but would be more unpredictable and likely result in poorer performance. While this infrastructure doubles the number of API calls, there are two clear functions that can be handled by individual agents.

Evaluation

The evaluation framework was built on Braintrust, an LLM evaluation platform. All 29 test cases were run against a pre-seeded user profile stored in Mem0, with 10 profiles covering varied demographics, dietary goals, and restriction scenarios seeded specifically for the multi-user evaluation.

Eight deterministic scorers were used – LLM scorers were avoided to reduce costs. The system completed all 29 test cases without errors. The tables below report aggregate scores across the full test suite and a per-category breakdown.

Test Category	Cases
Meal Planning	12
Dietary Restriction	6
Profile Update	4
Out of Scope	4
Shopping List	3
Total	**29**

Scorer	Average	Min	Max	n
AllergenSafety	1.000	1.00	1.00	5
NoError	1.000	1.00	1.00	29
ShoppingList	1.000	1.00	1.00	3
PlanCompleteness	0.984	0.833	1.00	21
ToolSelection	0.976	0.90	1.00	25
ScopeAwareness	0.931	0.00	1.00	29
CalorieTarget	0.476	0.172	1.00	18
Latency	0.422	0.00	1.00	29

Scorer	Meal Planning (n=12)	Dietary Restriction (n=6)	Shopping List (n=3)	Profile Update (n=4)	Out of Scope (n=4)
AllergenSafety	—	1.000	—	—	—
CalorieTarget	0.518	0.392	—	—	—
Latency	0.208	0.417	0.167	0.750	0.938
NoError	1.000	1.000	1.000	1.000	1.000
PlanCompleteness	0.986	0.972	1.000	—	—
ScopeAwareness	1.000	1.000	1.000	1.000	0.500
ShoppingList	—	—	1.000	—	—
ToolSelection	0.983	0.933	1.000	1.000	—

The system performed perfectly on safety-critical and structural correctness metrics. AllergenSafety scored 1.0 across all five allergen test cases, including cases with combined allergen and disliked-ingredient constraints. NoError was perfect across all 29 cases, and ShoppingList scored 1.0 on all three shopping list cases. PlanCompleteness (0.984) and ToolSelection (0.976) indicate that the orchestrator reliably routes requests to the correct tools and that the planning agent consistently produces complete meal plans.

Three metrics, however, scored much lower. CalorieTarget (0.476) checks whether total plan calories fall within ±15% of a user's computed target. This is due to the "BudgetBytes" portion sizes, as most meals are very low in calories. A tool was tested to scale up meals, but was not used for testing. Latency (0.422) also scored very low, as the 15-second threshold was not met by two or three sequential LLM calls plus Mem0 memory reads. Multi-day and shopping-list cases consistently fell into the 60 to 120 second range. Latency could be improved through parallel

tool execution, response caching, or routing simpler planning tasks to a faster model. Lastly, ScopeAwareness (0.931) – two of the four out-of-scope cases scored 0, bringing the out-of-scope category average to 0.5. In both cases the system provided contextually appropriate responses and avoided harmful advice, but did not include a sufficiently explicit refusal phrase to satisfy the scorer's pattern matching. This means that the scorer has to be readjusted rather than the system itself.

Responsible AI Considerations

A major safety issue associated with this project involves the increasingly prevalent issue of dietary restrictions, such as intolerances and allergies. Both the primary allergy enforcement and post-hoc meal plan audit rely on deterministic string matching to ensure recipes with allergens are excluded from the plan. It was built this way to reduce token usage, but in a deployed application, an additional agent could be added to detect potential violations using an LLM and clarify uncertainties with the user.

Ethical issues associated with an agentic nutrition plan are that it may lead to or feed into eating disorders. The system implicitly trusts the user to provide accurate profile information, but inherently allows for users to perpetuate unhealthy eating habits. Furthermore, the caloric and protein estimations are based on the Mifflin-St. Jeor equation, and may be inaccurate for those who do not conform to such a deterministic model. Lastly, the app is fundamentally based on large language models, not human judgement, and may misguide those who trust the app's recommendations. A deployed app must ensure that the user sees a licensed nutritionist or dietician for complications.

Other issues to consider involve cost estimations of ingredients, which may vary from place to place, and bias of certain cultural cuisines – to address these issues, most fields, including cost and cuisine preference, are customizable by the user.

Findings and Discussion

The results reflect a system that holds up where it matters most. AllergenSafety, NoError, and ShoppingList all scored perfectly, and both PlanCompleteness and ToolSelection exceeded 0.97. The lower scores on CalorieTarget and Latency are limitations, but are acceptable for the AI

agent focus of this project, as the former is a result of the recipe database (but could be solved by the system scaling up meals) and latter is a computer systems problem.
The ScopeAwareness edge cases were the most interesting finding. Two out-of-scope requests scored zero not because the system responded badly, but because it responded thoughtfully rather than with an explicit refusal. It avoided giving harmful advice while still being useful, which a pattern-matching scorer reads as a failure. That tension between being genuinely helpful and satisfying an evaluation rubric is worth keeping in mind when interpreting any binary scorer.
The clearest takeaway from building this is that separating concerns between agents pays off. The intake agent handles ambiguous conversation, the planning agent handles structured retrieval, and the audit layer checks constraints without touching an LLM. That division made the system easier to debug and more predictable overall.

Conclusion and Future Work

This project effectively proved the feasibility of an agentic AI meal planning system that can cater to the needs of a specific user. The system provides natural language interaction and cross-session persistence of memory. Perfect scores on many evaluators such as allergen safety and error rate prove the system's effectiveness.

Future work would involve making this a deployable product with proper authentication, database storage, server hosting, payment, and an improved user interface. Additional features could be implemented involving an agent that creates a shopping cart, or connects to an online grocery store MCP to automatically order groceries. More features such as LLM based allergy detection, meal size scaling, and validated nutritional values could also be added. Lastly, a time component could be implemented to track weight and caloric intake over time to adjust the nutrition plan dynamically.

Agentic AI is a powerful tool, but it is not effective without a strong underlying application. Most work (and most future work) involves building reliable non-AI features first, and then ensuring that the agentic AI infrastructure can support that feature, rather than focusing solely on AI features or agents. That being said, as agentic AI systems become more and more common,

they will provide greater accessibility, higher efficiency, and considerably improve the products, tools, and applications that people rely on every day.

References

Abbas, Adnan, et al. "'Having Lunch Now': Understanding How Users Engage with a Proactive Agent for Daily Planning and Self-Reflection." *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. 2026.

Angara, Prashanti, et al. "Foodie Fooderson a Conversational Agent for the Smart Kitchen." *CASCON*. Vol. 17. 2017.

Bilen, Ayşe Betül, Gülen Ecem Kalkan, and Hülya Yılmaz Önal. "Artificial Intelligence Diet Plans Underestimate Nutrient Intake Compared to Dietitians in Adolescents." *Frontiers in Nutrition* 13 (2026): 1765598.

Guo, Peiqi, et al. "From AI to the Table: A Systematic Review of ChatGPT's Potential and Performance in Meal Planning and Dietary Recommendations." *Dietetics* 4.1 (2025): 7.

Khamesian, Saman, et al. "NutriGen: Personalized Meal Plan Generator Leveraging Large Language Models to Enhance Dietary and Nutritional Adherence." *2025 47th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*. IEEE, 2025.

Papastratis, Ilias, et al. "AI Nutrition Recommendation Using a Deep Generative Model and ChatGPT." *Scientific Reports* 14.1 (2024): 14620.